

Formalização e prova da equivalência entre caminhos em uma grade cartesiana (*lattice paths*)

Victor Yan Martinez de Ávila (Matrícula 241032994)

Pedro do Nascimento Holanda (Matrícula 232005998)

Universidade de Brasília, Departamento de Ciência da Computação

Lógica Computacional 1, 2026/1, Prof. Flávio L. C. de Moura

28 de junho de 2026

Resumo

Este relatório desenvolve, no assistente de provas **Rocq** (compatível com o Coq), a formalização de caminhos em uma grade cartesiana discreta percorridos apenas com passos para a *direita* e para a *cima*. Modelamos um caminho como uma lista de passos e definimos a equivalência entre caminhos via o predicado indutivo **Permutation**. O resultado central, provado por Dedução Natural e indução estrutural, estabelece que caminhos equivalentes possuem o mesmo comprimento e o mesmo ponto final, tendo a comutatividade $RU \equiv UR$ como caso emblemático. A descrição é feita em linguagem natural; nos pontos centrais da prova, exibimos os blocos de código correspondentes para tornar evidente como cada ideia se materializa.

Sumário

1	Introdução e contexto	1
2	Objetivos e enunciado formal	2
3	Definições	2
4	Desenvolvimento das provas	3
4.1	O núcleo geométrico: passos comutam	3
4.2	Mesmo comprimento	3
4.3	Mesmo ponto final	4
4.4	Teorema principal e corolário	4
5	Decisões de modelagem e justificativas	4
6	Conclusão e extensões	5
	Referências	5

1 Introdução e contexto

Um *lattice path* (caminho em rede ou grade) é uma sequência de movimentos elementares sobre os pontos de coordenadas inteiras do plano. Consideramos a versão clássica: partindo de um ponto A , deseja-se alcançar um ponto B situado acima e à direita, usando apenas dois movimentos. O

passo para a direita (R) incrementa a coordenada horizontal e o *passo para cima* (U) incrementa a coordenada vertical.

Esses caminhos são objetos combinatórios fundamentais: o número de caminhos de $(0,0)$ a (m,n) é o coeficiente binomial $\binom{m+n}{m}$, pois cada caminho corresponde a uma escolha de quais dos $m+n$ passos são para a direita. Por trás desse fato existe uma observação simples, porém essencial, que é o foco da formalização: **reordenar os passos de um caminho não altera o destino alcançado**. O caso mínimo dessa propriedade é a comutatividade local $RU \equiv UR$, ou seja, ir uma casa à direita e depois uma para cima leva ao mesmo ponto que ir uma para cima e depois uma à direita.

O objetivo é tornar essa intuição um teorema verificado por máquina, exercitando os fundamentos pedidos pelo Departamento de Ciência da Computação da UnB: definições indutivas, Dedução Natural, indução estrutural sobre listas e sobre predicados, e equivalência de predicados.

2 Objetivos e enunciado formal

Queremos, em ordem: definir o tipo dos passos e o tipo dos caminhos; definir o efeito de um caminho sobre um ponto, que chamamos de *ponto final*; definir quando dois caminhos são *equivalentes*; provar que caminhos equivalentes têm o mesmo comprimento; e provar que caminhos equivalentes têm o mesmo ponto final.

O enunciado central, informalmente, diz: *se dois caminhos são equivalentes por uma permutação de passos, então têm o mesmo comprimento e o mesmo ponto final*. Os dois últimos objetivos são as duas metades desse enunciado, reunidas num teorema final. Todo o desenvolvimento se apoia na biblioteca padrão para listas e para o predicado de permutação.

3 Definições

Passos e caminhos. Um *passo* é definido como um tipo indutivo com exatamente dois construtores sem argumentos, uma enumeração, representando o movimento para a direita e o movimento para cima. Um *caminho* é, então, simplesmente uma *lista* de passos. A escolha por listas é deliberada: é o tipo indutivo sobre o qual o curso enfatiza a indução estrutural, e captura naturalmente a ideia de uma sequência ordenada de movimentos. Este é o tipo de dados em torno do qual todo o trabalho gira:

```
Inductive passo : Type :=
| Right : passo      (* deslocamento no eixo X (horizontal) *)
| Up      : passo.    (* deslocamento no eixo Y (vertical)   *)
```

```
Definition caminho := list passo.
```

Pontos e ponto final. Um *ponto* da grade é um par de números naturais (x,y) , em que x conta o deslocamento horizontal e y o vertical; a *origem* é o par $(0,0)$. O efeito de um passo sobre um ponto, dado por `aplica_passo`, soma 1 à primeira coordenada no caso R e 1 à segunda no caso U. O *destino* de um caminho a partir de um ponto é definido por recursão estrutural sobre a lista: o caminho vazio deixa o ponto inalterado, e um caminho com primeiro passo p e resto c' aplica p ao ponto e prossegue recursivamente sobre c' . Optamos por *executar* os passos um a um, em vez de apenas contar quantos passos de cada tipo existem, pois isso é mais fiel à ideia de percorrer a grade e mantém o enunciado independente de qualquer fórmula fechada para as coordenadas. Como o ponto final é o destino dessa execução, estas definições são o coração do que significa um caminho terminar em um ponto:

```
Definition aplica_passo (p : passo) (pt : ponto) : ponto :=
  match p with
```

```

| Right => (fst pt + 1, snd pt)
| Up    => (fst pt, snd pt + 1)
end.

```

```

Fixpoint destino (pt : ponto) (c : caminho) : ponto :=
  match c with
  | []      => pt
  | p :: c' => destino (aplica_passo p pt) c'
  end.

```

```

Definition ponto_final (c : caminho) : ponto := destino origem c.

```

Equivalência. Dizemos que dois caminhos são *equivalentes* quando um é uma *permutação* do outro. Em vez de reinventar essa noção, reutilizamos o predicado indutivo `Permutation` da biblioteca padrão, o mesmo apresentado na seção 7 do enunciado, cujas quatro regras de formação são: a permutação da lista vazia consigo mesma; a preservação de uma permutação ao acrescentar um mesmo elemento à frente de ambas as listas; a *troca* de dois elementos adjacentes; e a transitividade. A terceira regra, a troca de adjacentes, é exatamente a comutatividade local $RU \equiv UR$ que motiva o projeto, obtida diretamente como caso particular:

```

Definition equiv (c d : caminho) : Prop := Permutation c d.

(* R;U ~ U;R: caso direto de perm_swap, com a cauda da lista vazia. *)
Example comuta_RU : equiv [Right; Up] [Up; Right].
Proof. unfold equiv. apply perm_swap. Qed.

```

4 Desenvolvimento das provas

4.1 O núcleo geométrico: passos comutam

Antes dos teoremas principais, provamos um lema auxiliar que é o verdadeiro *heat point* do trabalho: aplicar dois passos quaisquer em sequência, a partir de um mesmo ponto, produz o mesmo resultado independentemente da ordem. A prova analisa os quatro casos possíveis, cada passo sendo R ou U, e em todos o ponto resultante é o mesmo, porque cada passo incrementa coordenadas independentes e a adição nessas coordenadas comuta. É este lema que justifica que a troca de dois passos adjacentes não altera o destino:

```

Lemma aplica_passo_comuta : forall (p q : passo) (pt : ponto),
  aplica_passo p (aplica_passo q pt) = aplica_passo q (aplica_passo p pt).
Proof.
  intros p q pt.
  destruct p; destruct q; destruct pt as [x y]; simpl; reflexivity.
Qed.

```

4.2 Mesmo comprimento

Provamos que caminhos equivalentes têm o mesmo comprimento por indução estrutural sobre a derivação da permutação, com um caso para cada regra de formação. No caso da lista vazia, ambos os comprimentos são zero. No caso do acréscimo de um mesmo elemento à frente, os dois comprimentos crescem de uma unidade e a hipótese de indução fecha o caso. No caso da troca de adjacentes, ambas as listas têm o mesmo comprimento, “dois mais o resto”. No caso da transitividade, encadeamos as duas hipóteses de indução. Observamos que a biblioteca padrão já oferece um lema pronto com esse conteúdo (`Permutation_length`); ainda assim, provamos

manualmente para exercitar explicitamente a indução sobre o predicado, como pede o caráter didático do trabalho.

4.3 Mesmo ponto final

Este é o teorema que reúne tudo, e seu bloco de prova é o segundo heat point. Provamos que caminhos equivalentes levam ao mesmo destino a partir de qualquer ponto de partida. A generalização sobre o ponto de partida é necessária para que a hipótese de indução seja forte o bastante; do contrário, o caso do acréscimo de um elemento à frente não fecharia. A prova é, de novo, por indução sobre a derivação da permutação. O caso da lista vazia é imediato. No caso do acréscimo de um mesmo passo à frente, reduzimos o destino um passo e aplicamos a hipótese de indução ao novo ponto de partida. No caso da troca de dois passos adjacentes, reduzimos dois passos de cada lado e usamos o lema geométrico de comutação para concluir que os destinos coincidem. No caso da transitividade, encadeamos as hipóteses:

```

Lemma equiv_mesmo_destino : forall c d : caminho,
  equiv c d -> forall pt : ponto, destino pt c = destino pt d.
Proof.
  intros c d H. unfold equiv in H.
  induction H; intros pt.
  - (* perm_nil *) reflexivity.
  - (* perm_skip *) simpl. apply IHPermutation.
  - (* perm_swap *) simpl. rewrite aplica_passo_comuta. reflexivity.
  - (* perm_trans*) rewrite IHPermutation1. apply IHPermutation2.
Qed.

```

Um detalhe técnico sustenta os casos do meio: como o passo é uma variável universal, `simpl` reduz `destino`, cujo casamento é sobre a lista que começa por `cons`, mas não desdobra `aplica_passo`, cujo casamento de padrão fica preso na variável. É exatamente isso que mantém a hipótese de indução e o lema de comutação aplicáveis.

4.4 Teorema principal e corolário

O teorema principal apenas reúne os dois resultados anteriores: caminhos equivalentes têm, simultaneamente, o mesmo comprimento e o mesmo ponto final. Como corolário imediato, dois caminhos equivalentes terminam no mesmo ponto final quando ambos partem da origem. Aplicamos o corolário ao caso emblemático RU e UR, confirmando formalmente que ambos terminam em (1, 1).

5 Decisões de modelagem e justificativas

- **Equivalência via Permutation.** Em vez de definir uma relação indutiva própria gerada pela troca local de passos, reutilizamos o predicado `Permutation` já existente. A regra `perm_swap` fornece exatamente a comutatividade desejada. Como há apenas dois símbolos, permutar elementos quaisquer coincide, neste domínio, com aplicar repetidamente a troca de passos adjacentes, de modo que a escolha não perde generalidade.
- **Ponto final por execução de passos.** Modelar o destino aplicando os passos um a um, em vez de contar R's e U's, é mais fiel ao conceito de percorrer a grade e mantém o teorema independente de fórmulas fechadas.
- **Lema de comutação como núcleo.** Isolar o fato de que passos comutam deixa explícito onde a geometria entra no argumento e torna a prova do ponto final curta.

- **Generalização do ponto de partida.** Enunciar o teorema do ponto final para um ponto de partida arbitrário, e não só a origem, é o que permite à indução prosseguir; recuperamos o caso da origem como corolário.
- **Estilo de provas.** Usamos apenas táticas elementares, sem automação aritmética, mantendo o desenvolvimento legível e alinhado à Dedução Natural.

6 Conclusão e extensões

Formalizamos caminhos em uma grade cartesiana com passos para a direita e para cima, definimos a equivalência entre caminhos via permutação de passos e provamos, com rigor verificado por máquina, que caminhos equivalentes têm o mesmo comprimento e o mesmo ponto final, com a comutatividade $RU \equiv UR$ como caso emblemático. O desenvolvimento é aceito pelo núcleo do Rocq e é totalmente construtivo: a consulta às hipóteses do teorema principal reporta “*Closed under the global context*”, ou seja, nenhum axioma adicional é utilizado.

Como extensões naturais, poderíamos: caracterizar o ponto final pela contagem de passos de cada tipo, provando que ele é igual ao par formado pelo número de R’s e pelo número de U’s; contar o número de caminhos entre dois pontos e relacioná-lo ao coeficiente binomial; ou enriquecer o conjunto de passos, por exemplo com passos diagonais, reavaliando quais propriedades de comutação se mantêm.

Referências

Referências

- [1] Flávio L. C. de Moura. *Projeto de Lógica Computacional 1 (2026/1)*. Universidade de Brasília, 2026.
- [2] The Rocq Prover. <https://rocq-prover.org>.
- [3] The Coq/Rocq Standard Library, módulo `Permutation`.